

INDEX

Name: Std: Sec: Roll No:

Sr.No	Date	Name of the Lesson	Page No.	Teacher's Sig
		Phase I :- Internet & Networking Fundamentals.		
(1)		DNS		
(2)		IP Address		
(3)		Ports		
(4)		HTTP, HTTPS, SSL, TLS (9,10)		
(5)		REST API		
(6)		cookies		
(7)		Session		
(8)		TWOT		
(9)		Network communication :-		
(10)		TCP (12) UDP (13) OSI Model (14) Websockets.		
(11)		(15) scalability Fundamentals :-		
(12)		performance vs scalability (16) latency vs throughput		
(13)		vertical scaling (17) Horizontal scaling		
(14)		(18) Traffic Management & performance		
(15)		(19) load balancer (20) caching (21) CDN		
(16)		(22) database Fundamentals & scaling		
(17)		(23) database indexing (24) replication (25) sharding		
(18)		(26) distributed system Fundamentals :-		
(19)		(27) CAP (28) consistent hashing		
(20)		(29) Messaging & Event system		
(21)		message queues (30) RabbitMQ (31) Kafka		
(22)		(32) Micro services		
(23)		(33) REST vs SOAP (34) API Gateway (35) service discovery		
(24)		(36) rate limiting (37) circuit breaker		
(25)		(38) Advanced Architecture patterns		
(26)		(39) CAPL (40) SaaS vs PaaS (41) High availability		
(27)		(42) Fault tolerance (43) Reverse proxy		
(28)		(44) Forward proxy (45) Nginx		

Reliability & Data consistency

- (43) Idempotency
- (44) event sourcing
- (45) distributed transactions
- (46) saga pattern

Advanced distributed system

- (47) distributed locking
- (48) bloom filters
- (49) leader election
- (50) consensus (raft basis)

storage system

(51) object storage

system design

client server architecture is a distributed computing model in which a client sends request for the service or resource and server process those requests and return the appropriate answers

Why it was invented?

Imagine 1000 user same data
in olden days

user A → own data

user B → own data

⋮ → ⋮

* Data duplication problem

* Maintenance problem

* security problem

to avoid these we use centralized server concept

Advantages

* centralized data

* Easy maintenance

* better security

* resource sharing

Disadvantage:

* single point of failure
if server down entire
application down

* client can't talk directly to the database.

client - load balancer - application server - cache - data base

client: entity that request resource

server: entity that provide resource

DNS :-

- * DNS (Domain Name System) is a hierarchical naming system that translates the human readable domain names into machine-readable IP addresses.
- * This allows users to access websites using easy-to-remember names instead of using numerical IP addresses.

Ex, when you open
instagram.com

- * your browser doesn't know Instagram is.
- * DNS finds the Insta IP address
- * only then can the browser connect to the Instagram server

DNS is also used for

- * load balancing
- * failover
- * traffic routing
- * CDN redirection

WWW → sub domain
google → domain
com → top level domain
• → root

companies like Google, Netflix and Amazon use advanced DNS strategies to direct users to the nearest data center

IP address :-

An IP address (Internet Protocol address) is a unique numerical identifier assigned to device connected to the network

It allows device to locate & communicate with each other

Why IP address is needed?

computer communicate using IP address
without IP address

data cannot reach the correct destination
devices can't identify each other

IP Types:-

Public IP

- * visible on internet
- * assigned by ISP
- * globally unique

Private IP

- * used inside local network
- * assigned by router
- * can repeat across network.

IP versions:-

IPv4

- * 32 bit
- * limited address
- * .

IPv6

- * 128 bit
- * massive address space.
- * :

Port:-

Port is a logical endpoint used to identify a specific service running on the computer

IP address = which computer?

Port number = which application?

common ports

- 80 (HTTP)
- 443 (HTTPS)
- 22 (SSH)
- 53 (DNS)
- 3306 (MySQL)

HTTP :-

HTTP is the language used by the browsers & servers to communicate

Browser — HTTP req — server — HTTP response — Browser

Methods
POST
GET
PUT
DELETE

Status codes
200 success
404 not found
500 server error
403 forbidden

- IP address tells you where to go
- Port tells you which service to use
- HTTP tells you how to communicate

HTTPS :-

HTTPS stands for the Hypertext transfer protocol secure

It is the secure version of HTTP

HTTP sends the data in plain text

HTTP it is encrypts the data before sending it over the internet.

Without HTTPS

browser

password = 12345

Internet

Server

anyone can intercept the traffic & read password and personal info

With HTTPS

Browser

encrypted data

Internet

server

no body can understand

* Look from on website url says https

https itself doesn't encrypt
https simply uses it (tls)

ssl:-

Secure socket layer is a protocol developed to provide secure communication over network

tls:-

Transport layer security is the successor to ssl and provide encrypted communication between client & server

TLS (handshake):-

when you open `https://google.com`

Browser & google perform a handshake

step 1:-

Browser

Hello google
can we communicate securely

step 2:-

Google sends certificate

I am google

Here's proof

step 3:- Browser verifies certificate

step 4:- Both create encryption keys

step 5:- secure communication begins.

Rest API

A Rest API (Representational state transfer application program interface) is an API that follows rest architecture principles and uses HTTP methods to enable communication between client & server

rest principles

① client - server architecture

client & server independent of each other

client handles user interface

server handles business logic & data.

Rest API

Rest API

spring boot server

② stateless communication

* The server doesn't store prior information about prior requests

* all are treated as completely independent requests

③ Resource based url

Resource are identified using url

④ uniform interface

Rest API use standard HTTP methods

⑤ cacheable responses

responses can be cached to improve performance and reduce server load

Cookie. (bcoz HTTP is stateless)

* A cookie is a small piece of data that stored in the users browser by a website to maintain state and remember user information across multiple HTTP requests.

login
 ↓
 server creates cookie (session id)
 ↓
 browser creates stores cookie
 ↓
 Future requests send cookie
 ↓
 server identifies user

session:-

* A session is a server side mechanism used to store information about a user across multiple requests.
 Each session is identified using unique session ID

login
 ↓
 session created by server
 ↓
 session ID generated by server
 ↓
 cookie sent
 ↓
 Future req carry session ID
 ↓
 server identifies user

JWT:-

* JSON web token is a compact, self contained token used for secure authentication and authorization between client and server

→ contains 3 parts of JWT

Header
 Payload
 Signature

* never store password in JWT bcoz JWT payload are encoded not encrypted

* JWT is stateless bcoz server don't store session data

login
|
generate jwt
|
store jwt
|
send jwt in requests
|
server verifies jwt

Networking :-

TCP

UDP

* Reliable

unreliable

connection-oriented

connectionless

ordered delivery

no ordering

retransmits packets

no retransmissions

slower

faster

banking

gaming

web applications

video calls

file transfer

live streaming

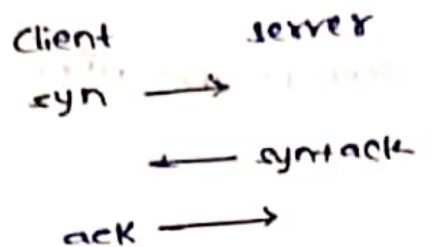
email

Which is best?

Both are best we should know when to use which protocol,

TCP three way handshake

TCP three way handshake is the protocol used to establish a reliable connection between client and server before data transfer begins.



connection established

After this

HTTP request

HTTP response

API calls can start

OSI Model (open systems interconnection model)

The OSI model is a conceptual framework that divides network communication into 7 layers.

7. Application
6. Presentation
5. Session
4. Transport
3. Network
2. Data link
1. Physical

Please
do
not
through
sausage
pizza
away

layer 7:- Application layer

This is the layer closest to the user

Ex:- HTTP

HTTPS

DNS

FTP

Purpose:-

Provide network services to applications

layer 6:- Presentation layer

Purpose:- Encryption

Decryption

compression

Ex:- SSL, TLS

Ex:- HTTPS password encryption

layer 5:- ~~transport~~ session layer

Purpose:- start communication

maintain communication

end communication

Ex:- A login session or video call session

layer 4:- Transport layer

Purpose:- Reliable delivery

Error checking

Port numbers

Ex:- TCP
UDP

TCP ensure data arrive correctly

layer 3:- Network layer

Purpose: routing,
IP addressing

Ex: IPv4
IPv6

Ex:- Finding google's server using its IP address

layer 2:- Data link layer

Purpose: communication with a local address.

Ex:- MAC address

Ethernet
wifi

layer 1:- physical layer

Purpose: physical transmission of data

Ex:- network cables

Fiber optics

wireless signals.

Web socket:-

* provides persistent connection

* support two way connection

* Enables real time applications

* connection remain open

used in whatsapp app

Discord

online games

stock trading apps

HTTP = Ask & receive

keep talking

scalability :-

scalability is the ability of a system to handle increasing loads without significant performance

degradation
performance :-
It focus on

- * speed
- * response time
- * efficiency

measures how fast the system response under a given workload.

Performance $\neq$ speed

performance = latency, throughput

latency

* Time taken for one request

* measured in msec

low is better

throughput

* No. of requests handled in unit time

* measured in req/sec

* high is better

vertical scaling

* Increase CPU, RAM storage

* uses one powerful server

* simple but limited

8GB RAM $\rightarrow$ 16GB RAM

CPU

2.

$\propto$ = bigger server

Horizontal scaling

* Add more servers

* distribute traffic

* Highly scalable

1 server $\rightarrow$ 10 servers

H1 = more servers

load balancer

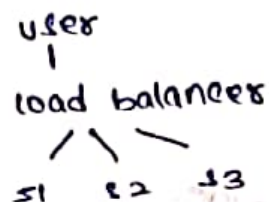
load balancer

distributes incoming request across the servers

Prevent server overload

Improve scalability & availability

Architecture :-



Algorithm:-

Round Robin

least connection (least server get request)

IP Hash (same user same server)

Health check:-

s1, s2, s3

↓
Fail

then s1, s3 get request

Benefits:- scalability

High availability

Better performance

load balancer = traffic police of server.

caching

caching

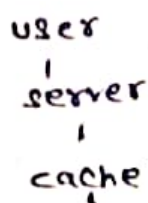
store frequently used data

improve response time

reduce database load

usually use memory (RAM)

Architecture



Cache hit = data found

Cache miss = data not found

Popular tools :- Redis, memcached

H2
 ↓
 H2
 ↓
 caching

CDN (Content delivery network)

CDN is one of the biggest reason why

youtube video load fast

netflix streams fastly

insta img load quickly

website load fast world wide.

CDN

network of geographically distributed servers

stores copies of content near users

reduces latency

improves performance

reduces origin server load.

flow

user
 ↓
 nearest CDN
 ↓
 origin server

without CDN

-enali
 ↓
 usa
 ↓
 -enali

with CDN

-enali
 ↓
 hyd cdn
 ↓
 -enali

cache hit → Found in CN

cache miss → content not found in CDN
or

content fetched from nearest origin server

cache

* near server

* reduce database queries

* Improve backend performance

CDN

* near user

* reduce distance

* Improve content delivery speed

cache stores the frequently used data near the application to reduce database queries where it stores content near user to reduce network latency & improve delivery speed

Database Index:-

improved data retrieval speed

avoid full table scan

works like a book index

commonly implemented using B-tree

Benefits:-

Faster select queries but slower insert, update, delete

Faster searching

Faster filtering

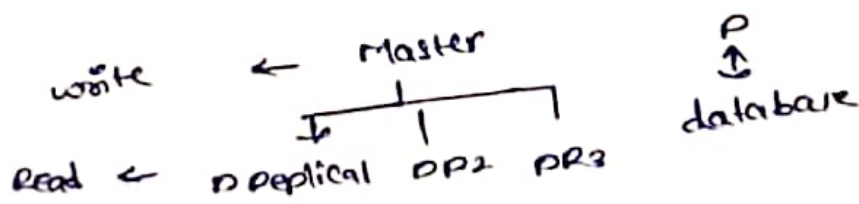
Drawbacks

extra storage

slow insert/delete

Database Replication:-

Database replication is the process of maintaining multiple copies of a database so that read requests can be distributed across the replicas and write requests handled by master database



Database Sharding:-

sharding is a database scaling technique in which data is horizontally partitioned across multiple database called shards to improve capacity and scalability.

Replication = same data on many databases
Read scaling

sharding = different data on many databases
Storage scaling
write scaling

Message Queue:-

A system that stores messages temporarily and processes them asynchronously

Producer → queue → consumer

with queue
user
|
Application
|
queue.

without queue
user
|
Application
|
email service
Inventory service
notification service
If any service slow
entire request slow

Ex:- user place order

order # 143 created

* message goes to queue

queue

* later worker reads it

worker
|
send email

RabbitMQ:

* Think to-do list
task done remove task

producer

queue

consumer

processed

deleted

use when

Reliable task processing
Background jobs
Email sending
Payment processing

Ex:- teacher giving tasks to student

purpose: task processing

common in

* E-commerce
* notification system

read note
still remaining

Kafka when
event streaming
- analytics
activity tracking

Ex: clg notice for
first 100 see
second 200 see
removed.

producer
↓
pic
↓
my customers
↓
Processed
↓
Parents still

on in
Netflix
uber
LinkedIn

1d example:-

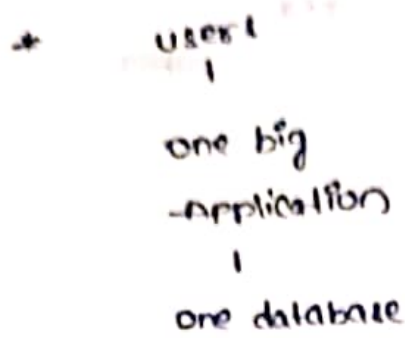
MQ
delivery app

er created
↓
then receives
↓
ask completed
↓
remove msg

Kafka
Instagram post created.

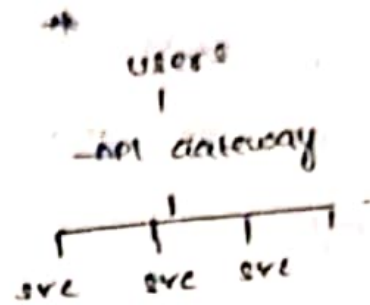
post created
↓
many services
need it
↓
Feed
Analytics
notification
↓
All read same event
↓
msg remain stored

Monolith



- * single application
- * single deployment
- * easier to start
- * hard to scale

Micro services

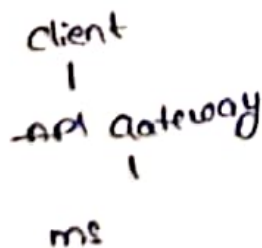


- * many small services
- * independent deployment
- * independent scaling
- * more complex

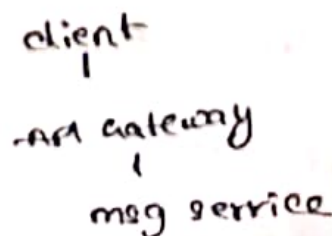
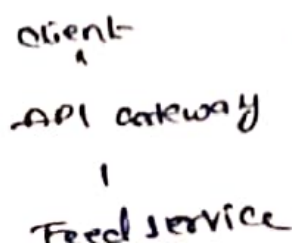
API Gateway

API Gateway

- * single entry point for clients
- * routes request to correct micro service
- * handles authentication
- * perform rate limiting
- * centralize logging



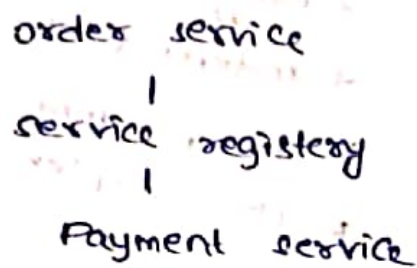
ex: Instagram



Service Discovery :-

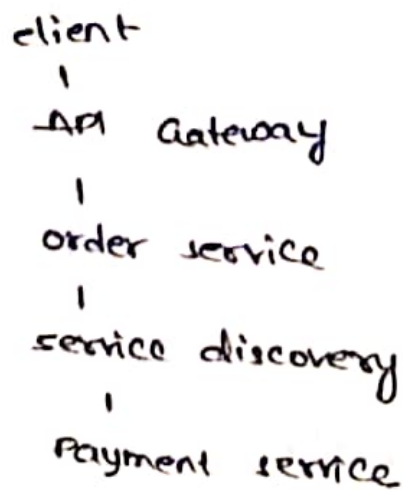
Service Discovery is a mechanism that help micro services find and communicate with each other automatically

Instead of hardcoding IP addresses
service register themselves in a registry



Ex:- payment service starts

- ② registers itself payment service IP = 10.0.0.08 stored in registry
 - ③ order service wants payment service
 - ④ order service ask registry
 - ⑤ registry replies 10.0.0.08
 - ⑥ communication starts
- Service registry:- A database of service names & locations



Tools:-
Eureka (popular in springboot ecosystem)
consul

Rate limiting &

Rate limiting

restricts requests per user/client
Protect system from abuse
improve reliability

Algo:
Fixed window
sliding window
Token bucket

status code : 429 (too many requests)

Circuit Breaker &

A circuit breaker is a design pattern that stops repeated requests to a failing service
It prevent cascading failure across the system

without circuit breaker

order service
↓

Retry Retry --
↓

Payment service(x)

resources get exhausted
eventually

order service(x) too

with circuit breaker

order service
|

circuit breaker
|

Payment service

3 states
① ~~open~~ ^{closed} state
req allowed

② open state
too many failure detected
circuit breaker
opens

order service
x
payment service

no req sent
immediate failure returned

Half open state

After some time

try again

one test req sent

if success $\rightarrow$ closed state

if failure $\rightarrow$ open state

not cascading failure (one failure doesn't spread)

failure response

load balancer

$\downarrow$

API gateway

$\downarrow$

micro services

$\downarrow$

service discovery

$\downarrow$

Rate limiting

$\downarrow$

circuit breaker

Cons:-

stands for command query responsibility system

separate Read & write

commands = write operation

like create post

create user

update profile

query = Read operations

like view post

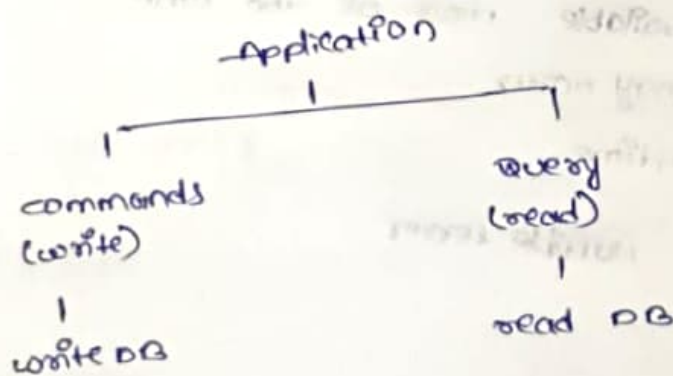
view feed

search user

read > write
1 million > 1 post
view

Improves scalability & performance

common in large scale systems like amazon, insta.



disadv

Adv

better per.

independent scaling

optimized read/write

Handle large traffic

more complexity

data syn needed

eventual consistency

higher maintainence

SQL

VS

NoSQL

relational database
 use tables
 fixed schema
 support ACID
 Best for transactions

mysql, postgresql

non relational is
 flexible schema
 easy horizontal scale
 best for large-scale
 distributed systems

casandra, dynamoDB
 MongoDB

Availability

service remain available most of the time

small down-time may occur

Focus: minimize downtime

Ex) load balancer + multiple server

99.1%

2.65 days downtime/year

99.91%

8.76 hrs/year

99.99%

52 min/year

99.999%

5 min/year

This is called Five nines availability

easy analogy: car tire puncture, you stop change tire
 continue
 small downtime

2)

e-commerce

fault tolerance:

system continue despite failures

little or no interruption

Goal: survive failures

Ex: primary + backup components

easy analogy: Airplane engine fails

second engine work instantly

no downtime

Ex: Banking system

Forward proxy

Reverse proxy

client

user

Forward proxy

Reverse proxy

Internet

Backend server

Hide client

Hide servers

By pass restrictions

load balancing

security

ssl termination

caching

Nginx:-

A high performance software used as
web server

reverse proxy

load balancer

cache

ssl terminator

Benefit:- fast
scalable
efficient
secure

users

Nginx

Application
servers

Idempotency

same request repeated multiple times
same final result
Event duplicated
essential for payment system

Ex:-
request
+
retry
+
retry
+

= one payment only

Rate limiting - limits requests

Circuit breaker - stop calls to failing services

Idempotency - makes retrieval safe.

Event sourcing

Store events, not current state

Events are immutable

current state is rebuilt from events

Ex:-

instead of balance = ₹1500

store

created 1000

deposited 500

Benefit

Full history

Audit trail

Recovery

Drawback

More complexity

Distributed transactions & saga pattern

Distributed transaction

A transaction that spans multiple services or DB

ex:-

order service

payment service

All participate in one business ops

Traditional ACID transactions don't work well across multiple services

BEGIN

UPDATE

COMMIT

ROLLBACK

DB

diff DB

diff servers

saga pattern

A saga is a sequence of local transactions

each service

① complete its transactions

② publish an event

③ next service continues

if a step fails

compensation actions

(undo peer work)

order created

↓

payment success

↓

inventory updated

↓

order confirmed

Failed Flow:

order created ✓

payment success ✓

inventory failed ✗

compensation starts

refund payment

cancel order

Distributed locking

prevent concurrent access to shared resources
ensure data consistency
common in booking & payment system

Flow:-

Acquire lock

↓

Perform ops

↓

Release lock

Race condⁿ solved

Boom filter

Fast existence checker
memory effⁿ
used before database lookup

Answer

Definitely not present
May be present

Use case

tinyurl
database
caches
web crawlers

leader election:-

select one node as leader
other node become followers
leader coordinates the system
new leader elected on failure

leader

leader fails

Followers

↓

election starts

↓

new leader selected

Benefits

coordination, consistency, fault recovery

consensus is the process by which multiple nodes in a distributed system agree on the same information or data value

suppose there are 3 servers

- A
- B
- C

who is the leader?
 what is the correct data?
 { All servers must agree on

Why consensus needed?

distributed system face

- network failure
- server crashes
- message delay

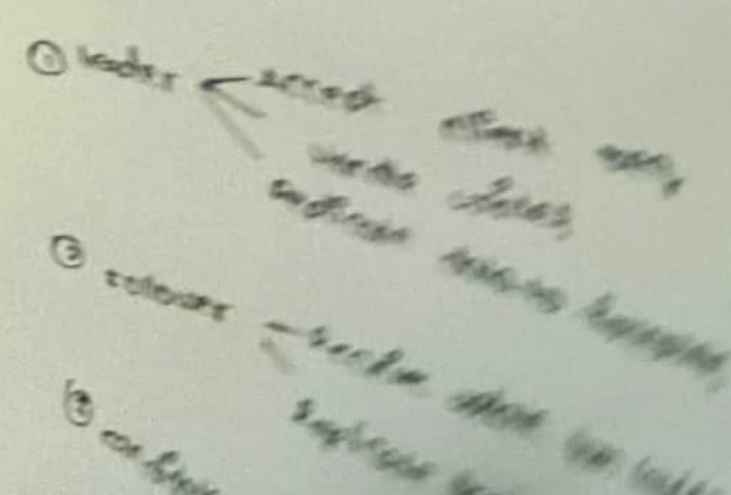
even during these failures, all servers must maintain same state

Raft is the popular consensus algorithm

used in kubernetes (etc)

- core
- distributed OS

all roles



quorum is the minimum no. of nodes required
agree on decision

$$m = n/2 + 1$$

object storage:

Object storage stores unstructured data
such as img, videos and documents as objects
and provide high scalable file storage

Popular services

→ Amazon S3

→ google cloud storage